

第16章 RCC—使用 HSE/HSI 配置时钟

本章参考资料：《STM32F4xx 中文参考手册》RCC 章节。

学习本章时，配合《STM32F4xx 中文参考手册》RCC 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

RCC：reset clock control 复位和时钟控制器。本章我们主要讲解时钟部分，特别是要着重理解时钟树，理解了时钟树，F407 的一切时钟的来龙去脉都会了如指掌。

16.1 RCC 主要作用—时钟部分

设置系统时钟 SYSCLK、设置 AHB 分频因子（决定 HCLK 等于多少）、设置 APB2 分频因子（决定 PCLK2 等于多少）、设置 APB1 分频因子（决定 PCLK1 等于多少）、设置各个外设的分频因子；控制 AHB、APB2 和 APB1 这三条总线时钟的开启、控制每个外设的时钟的开启。对于 SYSCLK、HCLK、PCLK2、PCLK1 这四个时钟的配置一般是：

$HCLK = SYSCLK = PLLCLK = 168M$, $PCLK1 = HCLK/2 = 84M$, $PCLK1 = HCLK/4 = 42M$ 。

这个时钟配置也是库函数的标准配置，我们用的最多的就是这个。

16.2 RCC 框图剖析—时钟树

时钟树单纯讲理论的话会比较枯燥，如果选取一条主线，并辅以代码，先主后次讲解的话会很容易，而且记忆还更深刻。我们这里选取库函数时钟系统时钟函数：

SetSysClock(); 以这个函数的编写流程来讲解时钟树，这个函数也是我们用库的时候默认的系统时钟设置函数。该函数的功能是利用 HSE 把时钟设置为： $HCLK = SYSCLK = PLLCLK = 168M$, $PCLK1 = HCLK/2 = 84M$, $PCLK1 = HCLK/4 = 42M$ 下面我们就以这个代码的流程为主线，来分析时钟树，对应的是图中的黄色部分，代码流程在时钟树中以数字的大小顺序标识。

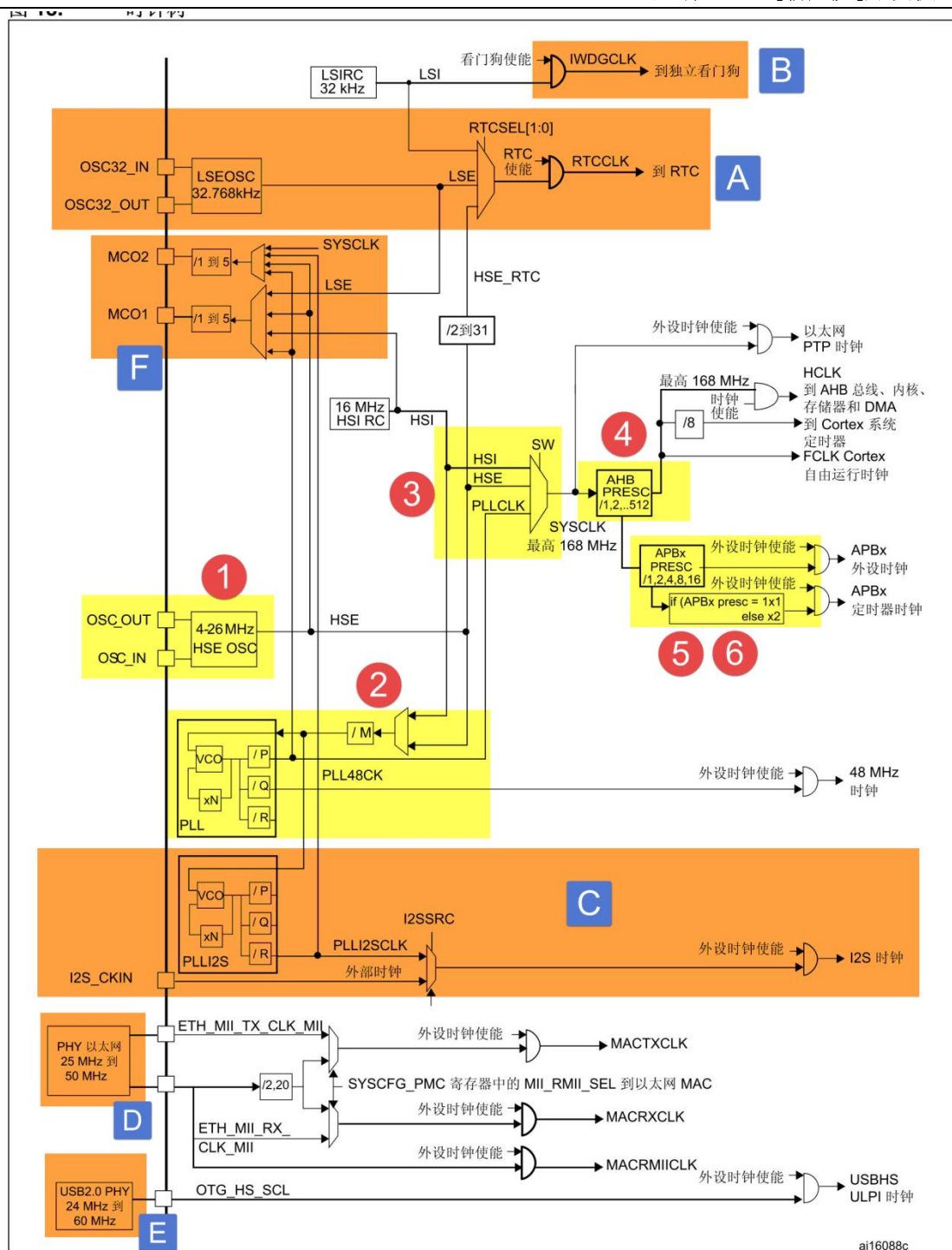


图 16-1 STM32F429 时钟树

16.2.1 系统时钟

1. ①HSE 高速外部时钟信号

HSE 是高速的外部时钟信号，可以由有源晶振或者无源晶振提供，频率从 4-26MHz 不等。当使用有源晶振时，时钟从 OSC_IN 引脚进入，OSC_OUT 引脚悬空，当选用无源晶振时，时钟从 OSC_IN 和 OSC_OUT 进入，并且要配谐振电容。HSE 我们使用 25M 的无源晶振。如果我们使用 HSE 或者 HSE 经过 PLL 倍频之后的时钟作为系统时钟 SYSCLK，当 HSE 故障时候，不仅 HSE 会被关闭，PLL 也会被关闭，此时高速的内部时钟信号 HSI 会作为备用的系统时钟，直到 HSE 恢复正常，HSI=16M。

2. ②锁相环 PLL

PLL 的主要作用是对时钟进行倍频，然后把时钟输出到各个功能部件。PLL 有两个，一个是主 PLL，另外一个专用的 PLLI2S，他们均由 HSE 或者 HSI 提供时钟输入信号。

主 PLL 有两路时钟输出，第一个输出时钟 PLLCLK 用于系统时钟，F407 里面最高是 168M，第二个输出用于 USB OTG FS 的时钟（48M）、RNG 和 SDIO 时钟（≤48M）。专用的 PLLI2S 用于生成精确时钟，给 I2S 提供时钟。

HSE 或者 HSI 经过 PLL 时钟输入分频因子 M（2~63）分频后，成为 VCO 的时钟输入，VCO 的时钟必须在 1~2M 之间，我们选择 HSE=25M 作为 PLL 的时钟输入，M 设置为 25，那么 VCO 输入时钟就等于 1M。

VCO 输入时钟经过 VCO 倍频因子 N 倍频之后，成为 VCO 时钟输出，VCO 时钟必须在 192~432M 之间。我们配置 N 为 336，则 VCO 的输出时钟等于 336M。如果要把系统时钟超频，就得在 VCO 倍频系数 N 这里做手脚。PLLCLK_OUTMAX = VCOCLK_OUTMAX/P_MIN = 432/2=216M，即 F407 最高可超频到 216M。

VCO 输出时钟之后有三个分频因子：PLLCLK 分频因子 p，USB OTG FS/RNG/SDIO 时钟分频因子 Q，分频因子 R（F446 才有，F407 没有）。p 可以取值 2、4、6、8，我们配置为 2，则得到 PLLCLK=168M。Q 可以取值 4~15，但是 USB OTG FS 必须使用 48M，Q=VCO 输出时钟 336/48=7。有关 PLL 的配置有一个专门的 RCC PLL 配置寄存器 RCC_PLLCFGR，具体描述看手册即可。

PLL 的时钟配置经过，稍微整理下可由如下公式表达：

$$VCOCLK_IN = PLLCLK_IN / M = HSE / 25 = 1M$$

$$VCOCLK_OUT = VCOCLK_IN * N = 1M * 336 = 336M$$

$PLLCLK_OUT = VCOCLK_OUT / P = 336 / 2 = 168M$

$USBCLK = VCOCLK_OUT / Q = 336 / 7 = 48$ 。。

3. ③系统时钟 SYSCLK

系统时钟来源可以是：HSI、PLLCLK、HSE，具体的由时钟配置寄存器 RCC_CFGR 的 SW 位配置。我们这里设置系统时钟：SYSCLK = PLLCLK = 168M。如果系统时钟是由 HSE 经过 PLL 倍频之后的 PLLCLK 得到，当 HSE 出现故障的时候，系统时钟会切换为 HSI=16M，直到 HSE 恢复正常为止。

4. ④AHB 总线时钟 HCLK

系统时钟 SYSCLK 经过 AHB 预分频器分频之后得到时钟叫 AHB 总线时钟，即 HCLK，分频因子可以是：[1,2,4, 8, 16, 64, 128, 256, 512]，具体的由时钟配置寄存器 RCC_CFGR 的 HPRE 位设置。片上大部分外设的时钟都是经过 HCLK 分频得到，至于 AHB 总线上的外设的时钟设置为多少，得等到我们使用该外设的时候才设置，我们这里只需粗线条的设置好 APB 的时钟即可。我们这里设置为 1 分频，即 HCLK=SYSCLK=168M。

5. ⑤APB2 总线时钟 HCLK2

APB2 总线时钟 PCLK2 由 HCLK 经过高速 APB2 预分频器得到，分频因子可以是：[1,2,4, 8, 16]，具体由时钟配置寄存器 RCC_CFGR 的 PPRE2 位设置。HCLK2 属于高速的总线时钟，片上高速的外设就挂载到这条总线上，比如全部的 GPIO、USART1、SPI1 等。至于 APB2 总线上的外设的时钟设置为多少，得等到我们使用该外设的时候才设置，我们这里只需粗线条的设置好 APB2 的时钟即可。我们这里设置为 2 分频，即 PCLK2 = HCLK / 2 = 84M。

6. ⑥APB1 总线时钟 HCLK1

APB1 总线时钟 PCLK1 由 HCLK 经过低速 APB 预分频器得到，分频因子可以是：[1,2,4, 8, 16]，具体由时钟配置寄存器 RCC_CFGR 的 PPRE1 位设置。HCLK1 属于低速的总线时钟，最高为 42M，片上低速的外设就挂载到这条总线上，比如 USART2/3/4/5、SPI2/3、I2C1/2 等。至于 APB1 总线上的外设的时钟设置为多少，得等到我们使用该外设的时候才设置，我们这里只需粗线条的设置好 APB1 的时钟即可。我们这里设置为 4 分频，即 PCLK1 = HCLK / 4 = 42M。

7. 设置系统时钟库函数

上面的 6 个步骤对应的设置系统时钟库函数具体见代码清单 16-1，为了方便阅读，已经把跟 407 不相关的代码删掉，把英文注释翻译成了中文，并把代码标上了序号，总共 6 个步骤。该函数是直接操作寄存器的，有关寄存器部分请参考数据手册的 RCC 的寄存器描述部分。

代码清单 16-1 设置系统时钟库函数

```
1  /*
2   * 使用 HSE 时，设置系统时钟的步骤
3   * 1、开启 HSE，并等待 HSE 稳定
4   * 2、设置 AHB、APB2、APB1 的预分频因子
5   * 3、设置 PLL 的时钟来源
6   *   设置 VCO 输入时钟 分频因子          m
7   *   设置 VCO 输出时钟 倍频因子          n
8   *   设置 PLLCLK 时钟分频因子            p
9   *   设置 OTG FS, SDIO, RNG 时钟分频因子 q
10  * 4、开启 PLL，并等待 PLL 稳定
11  * 5、把 PLLCK 切换为系统时钟 SYSCLK
12  * 6、读取时钟切换状态位，确保 PLLCLK 被选为系统时钟
13  */
14
15 #define PLL_M 25
16 #define PLL_N 336
17 #define PLL_P 2
18 #define PLL_Q 7
19 // 要超频的话，修改 PLL_N 这个宏即可，取值范围为：192~432。
20 void SetSysClock(void)
21 {
22
23     __IO uint32_t StartUpCounter = 0, HSEStatus = 0;
24
25     // ①使能 HSE
26     RCC->CR |= ((uint32_t)RCC_CR_HSEON);
27
28     // 等待 HSE 启动稳定
29     do {
30         HSEStatus = RCC->CR & RCC_CR_HSERDY;
31         StartUpCounter++;
32     } while ((HSEStatus==0)&&(StartUpCounter
33                                     !=HSE_STARTUP_TIMEOUT));
34
35     if ((RCC->CR & RCC_CR_HSERDY) != RESET) {
36         HSEStatus = (uint32_t)0x01;
37     } else {
38         HSEStatus = (uint32_t)0x00;
39     }
40
41     // HSE 启动成功
42     if (HSEStatus == (uint32_t)0x01) {
43         // 调压器电压输出级别配置为 1，以便在器件为最大频率
44         // 工作时使性能和功耗实现平衡
45         RCC->APB1ENR |= RCC_APB1ENR_PWREN;
46         PWR->CR |= PWR_CR_VOS;
47
48         // ②设置 AHB/APB2/APB1 的分频因子
49         // HCLK = SYSCLK / 1
50         RCC->CFGR |= RCC_CFGR_HPRE_DIV1;
```

```
51 // PCLK2 = HCLK / 2
52 RCC->CFGR |= RCC_CFGR_PPRE2_DIV2;
53 // PCLK1 = HCLK / 4
54 RCC->CFGR |= RCC_CFGR_PPRE1_DIV4;
55
56 // ③配置主 PLL 的时钟来源, 设置 M,N,P,Q
57 // Configure the main PLL
58 RCC->PLLCFGR = PLL_M|(PLL_N<<6)|
59               (((PLL_P >> 1) - 1) << 16) |
60               (RCC_PLLCFGR_PLLSRC_HSE) |
61               (PLL_Q << 24);
62
63 // ④使能主 PLL
64 RCC->CR |= RCC_CR_PLLON;
65
66 // 等待 PLL 稳定
67 while ((RCC->CR & RCC_CR_PLLRDY) == 0) {
68 }
69 /*-----*/
70 // 配置 FLASH 预取指, 指令缓存, 数据缓存和等待状态
71 FLASH->ACR = FLASH_ACR_PRFTEN
72             |FLASH_ACR_ICEN
73             |FLASH_ACR_DCEN
74             |FLASH_ACR_LATENCY_5WS;
75 /*-----*/
76
77 // ⑤选择主 PLLCLK 作为系统时钟源
78 RCC->CFGR &= (uint32_t)((uint32_t)~(RCC_CFGR_SW));
79 RCC->CFGR |= RCC_CFGR_SW_PLL;
80
81 // ⑥读取时钟切换状态位, 确保 PLLCLK 选为系统时钟
82 while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS )
83        != RCC_CFGR_SWS_PLL);
84 {
85 }
86 } else {
87 // HSE 启动出错处理
88 }
89 }
90
```

16.2.2 其他时钟

通过对系统时钟设置的讲解, 整个时钟树我们已经把握的有六七成, 剩下的时钟部分我们讲解几个重要的。

1. A、RTC 时钟

RTCCLK 时钟源可以是 HSE 1 MHz (HSE 由一个可编程的预分频器分频)、LSE 或者 LSI 时钟。选择方式是编程 RCC 备份域控制寄存器 (RCC_BDCR) 中的 RTCSEL[1:0] 位和 RCC 时钟配置寄存器 (RCC_CFGR) 中的 RTCPRE[4:0] 位。所做的选择只能通过复位备份域的方式修改。我们通常的做法是由 LSE 给 RTC 提供时钟, 大小为 32.768KHZ。LSE 由外接的晶体谐振器产生, 所配的谐振电容精度要求高, 不然很容易不起震。

2. B、独立看门狗时钟

独立看门狗时钟由内部的低速时钟 LSI 提供，大小为 32KHZ。

3. C、I2S 时钟

I2S 时钟可由外部的时钟引脚 I2S_CKIN 输入，也可由专用的 PLLI2SCLK 提供，具体的由 RCC 时钟配置寄存器 (RCC_CFGR) 的 I2SSCR 位配置。我们在使用 I2S 外设驱动 W8978 的时候，使用的时钟是 PLLI2SCLK，这样就可以省掉一个有源晶振。

4. D、PHY 以太网时钟

F407 要想实现以太网功能，除了有本身内置的 MAC 之外，还需要外接一个 PHY 芯片，常见的 PHY 芯片有 DP83848 和 LAN8720，其中 DP83848 支持 MII 和 RMII 接口，LAN8720 只支持 RMII 接口。野火 F407 开发板用的是 RMII 接口，选择的 PHY 芯片是 LAN8720。使用 RMII 接口的好处是使用的 IO 减少了一半，速度还是跟 MII 接口一样。当使用 RMII 接口时，PHY 芯片只需输出一路时钟给 MCU 即可，如果是 MII 接口，PHY 芯片则需要提供两路时钟给 MCU。

5. E、USB PHY 时钟

F407 的 USB 没有集成 PHY，要想实现 USB 高速传输的话，必须外置 USB PHY 芯片，常用的芯片是 USB3300。当外接 USB PHY 芯片时，PHY 芯片需要给 MCU 提供一个时钟。

外扩 USB3300 会占用非常多的 IO，跟 SDRAM 和 RGB888 的 IO 会复用的很厉害，鉴于 USB 高速传输用的比较少，野火 F407 霸天虎就没有外扩这个芯片。

6. F、MCO 时钟输出

MCO 是 microcontroller clock output 的缩写，是微控制器时钟输出引脚，主要作用是可以对外提供时钟，相当于一个有源晶振。F407 中有两个 MCO，由 PA8/PC9 复用所得。MCO1 所需的时钟源通过 RCC 时钟配置寄存器 (RCC_CFGR) 中的 MCO1PRE[2:0] 和 MCO1[1:0] 位选择。MCO2 所需的时钟源通过 RCC 时钟配置寄存器 (RCC_CFGR) 中的 MCO2PRE[2:0] 和 MCO2 位选择。有关 MCO 的 IO、时钟选择和输出速率的具体信息如下表所示：

时钟输出	IO	时钟来源	最大输出速率
MCO1	PA8	HSI、LSE、HSE、PLLCLK	100M

MCO2	PC9	HSE、PLLCLK、SYSCLK、PLLI2SCLK	100M
------	-----	-----------------------------	------

16.3 配置系统时钟实验

16.3.1 使用 HSE

一般情况下，我们都是使用 HSE，然后 HSE 经过 PLL 倍频之后作为系统时钟。F407 系统时钟最高为 168M，这个是官方推荐的最高的稳定时钟，如果你想铤而走险，也可以超频，超频最高能到 216M。

如果我们使用库函数编程，当程序来到 main 函数之前，启动文件：startup_stm32f40xxx.s 已经调用 SystemInit() 函数把系统时钟初始化成 168MHZ，SystemInit() 在库文件：system_stm32f4xx.c 中定义。如果我们想把系统时钟设置低一点或者超频的话，可以修改底层的库文件，但是为了维持库的完整性，我们可以根据时钟树的流程自行写一个。

16.3.2 使用 HSI

当 HSE 直接或者间接（HSE 经过 PLL 倍频）的作为系统时钟的时候，如果 HSE 发生故障，不仅 HSE 会被关闭，连 PLL 也会被关闭，这个时候系统会自动切换 HSI 作为系统时钟，此时 SYSCLK=HSI=16M，如果没有开启 CSS 和 CSS 中断的话，那么整个系统就只能在低速率运行，这是系统跟瘫痪没什么两样。

如果开启了 CSS 功能的话，那么可以当 HSE 故障时，在 CSS 中断里面采取补救措施，使用 HSI，重新设置系统频率为 168M，让系统恢复正常使用。但这只是权宜之计，并非万全之策，最好的方法还是要采取相应的补救措施并报警，然后修复 HSE。临时使用 HSI 只是为了把损失降低到最小，毕竟 HSI 较于 HSE 精度还是要低点。

F103 系列中，使用 HSI 最大只能把系统设置为 64M，并不能跟使用 HSE 一样把系统时钟设置为 72M，究其原因是 HSI 在进入 PLL 倍频的时候必须 2 分频，导致 PLL 倍频因子调到最大也只能到 64M，而 HSE 进入 PLL 倍频的时候则不用 2 分频。

在 F407 中，无论是使用 HSI 还是 HSE 都可以把系统时钟设置为 168M，因为 HSE 或者 HSI 在进入 PLL 倍频的时候都会被分频为 1M 之后再倍频。

还有一种情况是，有些用户不想用 HSE，想用 HSI，但是又不知道怎么用 HSI 来设置系统时钟，因为调用库函数都是使用 HSE，下面我们给出个使用 HSI 配置系统时钟例子，起个抛砖引玉的作用。

16.3.3 硬件设计

- 1、RCC
- 2、LED 一个

RCC 是单片机内部资源，不需要外部电路。通过 LED 闪烁的频率来直观的判断不同系统时钟频率对软件延时的效果。

16.3.4 软件设计

我们编写两个 RCC 驱动文件，bsp_clkconfig.h 和 bsp_clkconfig.c，用来存放 RCC 系统时钟配置函数。

1. 编程要点

- 1、开启 HSE/HSI，并等待 HSE/HSI 稳定
- 2、设置 AHB、APB2、APB1 的预分频因子
- 3、设置 PLL 的时钟来源，设置 VCO 输入时钟 分频因子 PLL_M，设置 VCO 输出时钟 倍频因子 PLL_N，设置 PLLCLK 时钟分频因子 PLL_P，设置 OTG FS,SDIO,RNG 时钟分频因子 PLL_Q。
- 4、开启 PLL，并等待 PLL 稳定
- 5、把 PLLCK 切换为系统时钟 SYSCLK
- 6、读取时钟切换状态位，确保 PLLCLK 被选为系统时钟

2. 代码分析

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。

使用 HSE 配置系统时钟

代码清单 16-2 使用 HSE 配置系统时钟

```
1 /*
2  * 使用 HSE 时，设置系统时钟的步骤
3  * 1、开启 HSE，并等待 HSE 稳定
4  * 2、设置 AHB、APB2、APB1 的预分频因子
5  * 3、设置 PLL 的时钟来源
6  * 设置 VCO 输入时钟 分频因子          m
7  * 设置 VCO 输出时钟 倍频因子          n
```

```
8 * 设置 PLLCLK 时钟分频因子 p
9 * 设置 OTG FS,SDIO,RNG 时钟分频因子 q
10 * 4、开启 PLL，并等待 PLL 稳定
11 * 5、把 PLLCK 切换为系统时钟 SYSCLK
12 * 6、读取时钟切换状态位，确保 PLLCLK 被选为系统时钟
13 */
14
15 /*
16 * m: VCO 输入时钟 分频因子，取值 2~63
17 * n: VCO 输出时钟 倍频因子，取值 192~432
18 * p: PLLCLK 时钟分频因子，取值 2, 4, 6, 8
19 * q: OTG FS,SDIO,RNG 时钟分频因子，取值 4~15
20 * 函数调用举例，使用 HSE 设置时钟
21 * SYSCLK=HCLK=168M, PCLK2=HCLK/2=84M, PCLK1=HCLK/4=42M
22 * HSE_SetSysClock(25, 336, 2, 7);
23 * HSE 作为时钟来源，经过 PLL 倍频作为系统时钟，这是通常的做法
24
25 * 系统时钟超频到 216M 爽一下
26 * HSE_SetSysClock(25, 432, 2, 9);
27 */
28 void HSE_SetSysClock(uint32_t m, uint32_t n, uint32_t p, uint32_t q)
29 {
30     __IO uint32_t HSEStartUpStatus = 0;
31
32     // 使能 HSE，开启外部晶振，野火 F407 使用 HSE=25M
33     RCC_HSEConfig(RCC_HSE_ON);
34
35     // 等待 HSE 启动稳定
36     HSEStartUpStatus = RCC_WaitForHSEStartUp();
37
38     if (HSEStartUpStatus == SUCCESS) {
39         // 调压器电压输出级别配置为 1，以便在器件为最大频率
40         // 工作时使性能和功耗实现平衡
41         RCC->APB1ENR |= RCC_APB1ENR_PWREN;
42         PWR->CR |= PWR_CR_VOS;
43
44         // HCLK = SYSCLK / 1
45         RCC_HCLKConfig(RCC_SYSCLK_Div1);
46
47         // PCLK2 = HCLK / 2
48         RCC_PCLK2Config(RCC_HCLK_Div2);
49
50         // PCLK1 = HCLK / 4
51         RCC_PCLK1Config(RCC_HCLK_Div4);
52
53         // 如果要超频就得在这里下手啦
54         // 设置 PLL 来源时钟，设置 VCO 分频因子 m，设置 VCO 倍频因子 n，
55         // 设置系统时钟分频因子 p，设置 OTG FS,SDIO,RNG 分频因子 q
56         RCC_PLLConfig(RCC_PLLSource_HSE, m, n, p, q);
57
58         // 使能 PLL
59         RCC_PLLCmd(ENABLE);
60
61         // 等待 PLL 稳定
62         while (RCC_GetFlagStatus(RCC_FLAG_PLLRDY) == RESET) {
63         }
64
65         /*-----*/
66         // 配置 FLASH 预取指，指令缓存，数据缓存和等待状态
67         FLASH->ACR = FLASH_ACR_PRFTEN
68                     | FLASH_ACR_ICEN
69                     | FLASH_ACR_DCEN
70                     | FLASH_ACR_LATENCY_5WS;
71         /*-----*/
72     }
```

```
73 // 当 PLL 稳定之后, 把 PLL 时钟切换为系统时钟 SYSCLK
74 RCC_SYSCLKConfig(RCC_SYSCLKSource_PLLCLK);
75
76 // 读取时钟切换状态位, 确保 PLLCLK 被选为系统时钟
77 while (RCC_GetSYSCLKSource() != 0x08) {
78 }
79 } else {
80 // HSE 启动出错处理
81
82 while (1) {
83 }
84 }
85 }
```

这个函数采用库函数编写, 代码理解参考注释即可。函数有 4 个形参 m、n、p、q, 具体说明见表格 16-1。

表格 16-1 PLL 配置因子

形参	形参说明	取值范围
m	VCO 输入时钟 分频因子	2~63
n	VCO 输出时钟 倍频因子	192~432
p	PLLCLK 时钟分频因子	2/4/6/8
q	OTG FS,SDIO,RNG 时钟分频因子	4~15

HSE 我们使用 25M, 参数 m 我们一般也设置为 25, 所以我们需要修改系统时钟的时候只需要修改参数 n 和 p 即可, $SYSCLK=PLLCLK=HSE/m*n/p$ 。

函数调用举例: HSE_SetSysClock(25, 336, 2, 7) 把系统时钟设置为 168M, 这个跟库里面的系统时钟配置是一样的。HSE_SetSysClock(25, 432, 2, 9)把系统时钟设置为 216M, 这个是超频, 要慎用。

使用 HSI 配置系统时钟

代码清单 16-3 使用 HSI 配置系统时钟

```
1 /*
2  * 使用 HSI 时, 设置系统时钟的步骤
3  * 1、开启 HSI , 并等待 HSI 稳定
4  * 2、设置 AHB、APB2、APB1 的预分频因子
5  * 3、设置 PLL 的时钟来源
6  * 设置 VCO 输入时钟 分频因子 m
7  * 设置 VCO 输出时钟 倍频因子 n
8  * 设置 SYSCLK 时钟分频因子 p
9  * 设置 OTG FS,SDIO,RNG 时钟分频因子 q
10 * 4、开启 PLL, 并等待 PLL 稳定
11 * 5、把 PLLCLK 切换为系统时钟 SYSCLK
12 * 6、读取时钟切换状态位, 确保 PLLCLK 被选为系统时钟
13 */
14
15 /*
16 * m: VCO 输入时钟 分频因子, 取值 2~63
```

```
17 * n: VCO 输出时钟 倍频因子, 取值 192~432
18 * p: PLLCLK 时钟分频因子, 取值 2, 4, 6, 8
19 * q: OTG FS, SDIO, RNG 时钟分频因子, 取值 4~15
20 * 函数调用举例, 使用 HSI 设置时钟
21 * SYSCLK=HCLK=168M, PCLK2=HCLK/2=84M, PCLK1=HCLK/4=42M
22 * HSI_SetSysClock(16, 336, 2, 7);
23 * HSE 作为时钟来源, 经过 PLL 倍频作为系统时钟, 这是通常的做法
24
25 * 系统时钟超频到 216M 爽一下
26 * HSI_SetSysClock(16, 432, 2, 9);
27 */
28
29 void HSI_SetSysClock(uint32_t m, uint32_t n, uint32_t p, uint32_t q)
30 {
31     __IO uint32_t HSISetupStatus = 0;
32
33     // 把 RCC 外设初始化成复位状态
34     RCC_DeInit();
35
36     //使能 HSI, HSI=16M
37     RCC_HSICmd(ENABLE);
38
39     // 等待 HSI 就绪
40     HSISetupStatus = RCC->CR & RCC_CR_HSIIRDY;
41
42     // 只有 HSI 就绪之后则继续往下执行
43     if (HSISetupStatus == RCC_CR_HSIIRDY) {
44         // 调压器电压输出级别配置为 1, 以便在器件为最大频率
45         // 工作时使性能和功耗实现平衡
46         RCC->APB1ENR |= RCC_APB1ENR_PWREN;
47         PWR->CR |= PWR_CR_VOS;
48
49         // HCLK = SYSCLK / 1
50         RCC_HCLKConfig(RCC_SYSCLK_Div1);
51
52         // PCLK2 = HCLK / 2
53         RCC_PCLK2Config(RCC_HCLK_Div2);
54
55         // PCLK1 = HCLK / 4
56         RCC_PCLK1Config(RCC_HCLK_Div4);
57
58         // 如果要超频就得在这里下手啦
59         // 设置 PLL 来源时钟, 设置 VCO 分频因子 m, 设置 VCO 倍频因子 n,
60         // 设置系统时钟分频因子 p, 设置 OTG FS, SDIO, RNG 分频因子 q
61         RCC_PLLConfig(RCC_PLLSource_HSI, m, n, p, q);
62
63         // 使能 PLL
64         RCC_PLLCmd(ENABLE);
65
66         // 等待 PLL 稳定
67         while (RCC_GetFlagStatus(RCC_FLAG_PLLRDY) == RESET) {
68         }
69
70         /*-----*/
71         // 配置 FLASH 预取指, 指令缓存, 数据缓存和等待状态
72         FLASH->ACR = FLASH_ACR_PRFTEN
73                     | FLASH_ACR_ICEN
74                     | FLASH_ACR_DCEN
75                     | FLASH_ACR_LATENCY_5WS;
76         /*-----*/
77
78         // 当 PLL 稳定之后, 把 PLL 时钟切换为系统时钟 SYSCLK
79         RCC_SYSCLKConfig(RCC_SYSCLKSource_PLLCLK);
80
81         // 读取时钟切换状态位, 确保 PLLCLK 被选为系统时钟
```

```
82     while (RCC_GetSYSCLKSource() != 0x08) {  
83     }  
84 } else {  
85     // HSI 启动出错处理  
86     while (1) {  
87     }  
88 }  
89 }
```

这个函数采用库函数编写，代码理解参考注释即可。函数有 4 个形参 m、n、p、q，具体说明见表格 16-2。

表格 16-2 PLL 配置因子

形参	形参说明	取值范围
m	VCO 输入时钟 分频因子	2~63
n	VCO 输出时钟 倍频因子	192~432
p	PLLCLK 时钟分频因子	2/4/6/8
q	OTG FS,SDIO,RNG 时钟分频因子	4~15

HSI 为 16M，参数 m 我们一般也设置为 16，所以我们需要修改系统时钟的时候只需要修改参数 n 和 p 即可， $SYSCLK=PLLCLK=HSI/m*n/p$ 。

函数调用举例：HSI_SetSysClock(16, 336, 2, 7) 把系统时钟设置为 168M，这个跟库里面的系统时钟配置是一样的。HSI_SetSysClock(16, 432, 2, 9) 把系统时钟设置为 216M，这个是超频，要慎用。

软件延时

```
1 void Delay(__IO uint32_t nCount)  
2 {  
3     for (; nCount != 0; nCount--);  
4 }
```

软件延时函数，使用不同的系统时钟，延时时间不一样，可以通过 LED 闪烁的频率来判断。

MCO 输出

在 F407 中，PA8/PC9 可以复用为 MCO1/2 引脚，对外提供时钟输出，我们也可以利用示波器监控该引脚的输出来判断我们的系统时钟是否设置正确。

代码 9 MCO GPIO 初始化

```
1 // MCO1 PA8 GPIO 初始化  
2 void MCO1_GPIO_Config(void)  
3 {  
4     GPIO_InitTypeDef GPIO_InitStructure;  
5     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);  
6  
7     // MCO1 GPIO 配置  
8     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8;  
9     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;  
10    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;  
11    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
```

```
12     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
13     GPIO_Init(GPIOA, &GPIO_InitStructure);
14 }
15
16 // MCO2 PC9 GPIO 初始化
17 void MCO2_GPIO_Config(void)
18 {
19     GPIO_InitTypeDef GPIO_InitStructure;
20     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
21
22     // MCO2 GPIO 配置
23     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
24     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
25     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
26     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
27     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
28     GPIO_Init(GPIOC, &GPIO_InitStructure);
29 }
```

野火 F407 开发板中 PA8 与 PC9 都引出来了，在摄像头接口位置，都可以使用示波器来监控波形。

代码 10 MCO 输出时钟选择

```
1 // MCO1 输出 PLLCLK
2 RCC_MCO1Config(RCC_MCO1Source_PLLCLK, RCC_MCO1Div_1);
3
4 // MCO1 输出 SYSCLK
5 RCC_MCO2Config(RCC_MCO2Source_SYSCLK, RCC_MCO1Div_1);
```

我们初始化 MCO 引脚之后，可以直接调用库函数 `RCC_MCOxConfig()` 来选择 MCO 时钟来源，同时还可以分频，这两个参数的取值参考库函数说明即可。

主函数

代码清单 16-4 main 函数

```
1 // 使用 HSE 或者 HSI 配置系统时钟
2 #include "stm32f4xx.h"
3 #include "./rcc/bsp_clkconfig.h"
4 #include "./led/bsp_led.h"
5
6 void Delay(__IO u32 nCount);
7
8 /**
9  * @brief 主函数
10  * @param 无
11  * @retval 无
12  */
13 int main(void)
14 {
15     // 程序来到 main 函数之前，启动文件：startup_stm32f4xx.s 已经调用
16     // SystemInit() 函数把系统时钟初始化成 168MHz
17     // SystemInit() 在 system_stm32f4xx.c 中定义
18     // 如果用户想修改系统时钟，可自行编写程序修改
19     // 重新设置系统时钟，这时候可以选择使用 HSE 还是 HSI
20
21     // 使用 HSE，配置系统时钟为 168M
22     HSE_SetSysClock(25, 336, 2, 7);
23
24     // 系统时钟超频到 216M 爽一下，最高是 216M，别往死里整
25     // HSE_SetSysClock(25, 432, 2, 9);
26 }
```

```
27 // 使用 HSI, 配置系统时钟为 168M
28 // HSI_SetSysClock(16, 336, 2, 7);
29
30 // LED 端口初始化
31 LED_GPIO_Config();
32
33 // MCO GPIO 初始化
34 MCO1_GPIO_Config();
35 MCO2_GPIO_Config();
36
37 // MCO1 输出 PLLCLK
38 RCC_MCO1Config(RCC_MCO1Source_PLLCLK, RCC_MCO1Div_1);
39
40 // MCO2 输出 SYSCLK
41 RCC_MCO2Config(RCC_MCO2Source_SYSCLK, RCC_MCO1Div_1);
42
43 while (1) {
44     LED1( ON ); // 亮
45     Delay(0x0FFFFFF);
46     LED1( OFF ); // 灭
47     Delay(0x0FFFFFF);
48 }
49 }
50
51
52 void Delay(__IO uint32_t nCount) //简单的延时函数
53 {
54     for (; nCount != 0; nCount--);
55 }
56
```

在主函数中，可以调用 HSE_SetSysClock()或者 HSI_SetSysClock()这两个函数把系统时钟设置成各种常用的时钟，然后通过 MCO 引脚监控，或者通过 LED 闪烁的快慢体验不同的系统时钟对同一个软件延时函数的影响。

16.3.5 下载验证

把编译好的程序下载到开发板，可以看到设置不同的系统时钟时，LED 闪烁的快慢不一样。更精确的数据我们可以用示波器监控 MCO 引脚看到。